# 技术部第二次技术分享会

Java 基础语法

姚雨森　技术部分管主席

2026-10-23

# 让程序主动“开口”

我们需要知道程序正在执行什么，可以让它把信息打印到控制台。

使用 System.out.println(...) 输出一行文字：

```
System.out.println("Hello, Java!");
```

双引号里面的文字会被打印出来，双引号本身不会显示。

控制台输出：

```
Hello, Java!
```

# 把输出语句写在 main 中

程序从 main 方法开始执行。先把输出语句写在它的大括号内：

```
public class Main {
    public static void main(String[] args) {
        System.out.println("First");
        System.out.println("Second");
    }
}
```

这里的两条语句**从上到下依次执行**，每次输出后换行：

```
First
Second
```

# 注释：给人看的说明

注释用于解释代码，注释中的内容不会被当作语句执行。

- 单行注释：`//` 后面直到本行末尾的内容。
- 多行注释：`/*` 与 `*/` 之间的内容，可以跨行。

下面的代码写在 `main` 中：

```
// Print a greeting.
System.out.println("Hello!"); // This is a comment.
/* The next statement is commented out.
System.out.println("Hidden");
*/
System.out.println("Goodbye!");
```

控制台依次输出 `Hello!` 和 `Goodbye!`，各占一行；`Hidden` 不会输出。

# 变量：保存程序需要的数据

程序有时需要做运算，也需要保存一些数据，比如数值、文本。

可以用**变量**保存这些数据，再通过变量名读取或修改它们。

下面的代码写在 main 中：

```
int score = 60;
System.out.println(score);

score = 70;
System.out.println(score);
```

- `int score = 60;`：声明整数变量 score，初始化为 60。
- `score = 70;`：重新赋值，把 score 中保存的值改为 70。
- 两次 println 依次输出 60 和 70，各占一行。

# 变量的声明与赋值

## 声明变量时赋值（初始化）

语法：　类型　变量名　=　初始值;

```
int score = 60;
```

## 也可以先声明，再赋值

声明：　类型　变量名;
赋值：　变量名　=　值;

```
int score;
score = 60;
System.out.println(score);
```

赋值时不用再写类型。

# 整数类型：能保存的范围不同

byte、short、int 和 long 都能保存整数。

它们占用的位数不同，能保存的整数范围也不同：

| 类型 | 位数 | 取值范围 | 声明并赋值 |
|---|---|---|---|
| byte | 8 | $-128 \sim 127$ | `byte level = 10;` |
| short | 16 | $-32768 \sim 32767$ | `short count = 1000;` |
| int | 32 | $-2^{31} \sim 2^{31}-1$ | `int score = 60;` |
| long | 64 | $-2^{63} \sim 2^{63}-1$ | `long total = 3000000000L;` |

- 一般使用 int 即可，范围约为负 21 亿到正 21 亿。
- 需要更大范围时使用 long；上例数字末尾的 L 表示 long。

# 浮点类型：保存带小数的数值

`float` 和 `double` 的取值范围与精度不同。

| 类型 | 位数 | 有限值范围（约） | 有效数字 |
| --- | --- | --- | --- |
| `float` | 32 | $-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$ | 约 7 位 |
| `double` | 64 | $-1.8 \times 10^{308} \sim 1.8 \times 10^{308}$ | 约 15–16 位 |

```
float temperature = 36.5F;
double height = 1.75;
```

- 一般使用 `double`；带小数点的数值默认是 `double`。
- 给 `float` 赋值时，上例在数字末尾加 `F`。
- 浮点数通常是近似值，不能精确表示范围内的每一个小数。

# 布尔类型：只有 true 和 false

boolean 只有两个值：true（真）和 false（假）。

```
boolean ready = true;
System.out.println(ready);

ready = false;
System.out.println(ready);
```

两次输出依次为 true 和 false，各占一行。

先记住它的写法，具体作用留到后面介绍。

# 字符类型：用单引号表示一个字符

char 可以保存一个英文字母，比如 A。

```
char letter = 'A';
System.out.println(letter);

letter = 'B';
System.out.println(letter);
```

- 字符使用**单引号**，如 'A'；不要写成双引号 "A"。
- 两次输出依次为 A 和 B，引号本身不会显示。

# 自动转换与强制转换

**自动转换**：如 int 转为 long 或 double，Java 会自动完成。

```
int count = 10;
double value = count;
System.out.println(value); // 10.0
```

**强制转换**：浮点数转整数，或整数变量转为范围更小的类型时，需要显式写出（目标类型）。

```
double price = 19.9;
int whole = (int) price;
System.out.println(whole); // 19
```

- 上例转为整数时舍去小数部分，不是四舍五入。
- 强制转换可能丢失信息；自动转换也不一定保留全部精度。

# 算术运算符：加、减、乘、除与取余

我们可以对数字做运算，把结果保存到变量中，或直接打印出来。

| 符号 | 运算 | 表达式示例 | 结果 |
|---|---|---|---|
| + | 加法 | 7 + 2 | 9 |
| - | 减法 | 7 - 2 | 5 |
| * | 乘法 | 7 * 2 | 14 |
| / | 除法 | 7 / 2 | 3 |
| % | 取余 | 7 % 2 | 1 |

```
int sum = 7 + 2;
System.out.println(sum);   // 9
```

% 求除法的余数，常被称为“取模”；这里先看正整数。

# 括号可以改变运算顺序

默认先计算乘法、除法和取余，再计算加法、减法。

**没有括号：先乘，再加**

```
int result = 2 + 3 * 4;
System.out.println(result); // 14
```

先算 3 * 4 得到 12，再算 2 + 12 得到 14。

**加上括号：先计算括号里面的部分**

```
result = (2 + 3) * 4;
System.out.println(result); // 20
```

先算 2 + 3 得到 5，再算 5 * 4 得到 20。

# 超出整数范围：结果可能变成负数

byte 的范围是 $-128 \sim 127$，无法保存 128。

```
byte number = 127;
System.out.println(number + 1); // 128
```

- byte 参与这里的加法时，先提升为 int，得到 128。
- 强制转回 byte 时只保留低 8 位，结果变为 $-128$。
- 转换不会自动检查范围或报错；应选择范围足够大的类型。

short 对应的边界是 32767：加 1 后转回 short 得到 $-32768$。

# 整数除法：先转换，再相除

两个整数相除，结果仍是整数，小数部分向零截断。

```
int total = 7;
int count = 2;
System.out.println(total / count);          // 3
System.out.println((double) total / count); // 3.5
```

想保留小数，先把其中一个操作数转成 double，再做除法。

**只把结果赋给 double，或除完再转换，都来不及：**

```
double result = total / count;
System.out.println(result);                 // 3.0
System.out.println((double) (total / count)); // 3.0
```

# 自增、自减与运算后赋值

**自增与自减**：++ 让变量加 1，-- 让变量减 1。

```
int count = 5;
count++; // count is now 6
count--; // count is now 5
```

**复合赋值**：把运算结果保存回原变量。以下以 int 变量为例：

| 简写 | 对应写法 |
| --- | --- |
| `count += 2;` | `count = count + 2;` |
| `count -= 2;` | `count = count - 2;` |
| `count *= 2;` | `count = count * 2;` |
| `count /= 2;` | `count = count / 2;` |
| `count %= 2;` | `count = count % 2;` |

# 比较运算：结果是 boolean

比较两个数值，结果是 `true` 或 `false`，可以保存到 `boolean` 变量中。

| 运算符 | 含义 | 示例 | 结果 |
|---|---|---|---|
| `==` | 等于 | `7 == 2` | `false` |
| `!=` | 不等于 | `7 != 2` | `true` |
| `>` | 大于 | `7 > 2` | `true` |
| `<` | 小于 | `7 < 2` | `false` |
| `>=` | 大于或等于 | `7 >= 7` | `true` |
| `<=` | 小于或等于 | `7 <= 2` | `false` |

```
boolean passed = 80 >= 60;
System.out.println(passed); // true
```

注意：`=` 是赋值，`==` 才是判断是否相等。

# 逻辑运算：组合或取反条件

逻辑运算的操作数和结果都是 boolean。

| 运算符 | 含义 | 结果为 true 的条件 |
| --- | --- | --- |
| && | 与 | 两边都为 true |
| \|\| | 或 | 至少一边为 true |
| ! | 非（取反） | 原来的值为 false |

```
int score = 80;
boolean inRange = (score >= 0) && (score <= 100); // true
boolean outside = (score < 0) || (score > 100);   // false
boolean failed = !(score >= 60);                  // false
```

# 字符串：保存文本信息

字符串可以理解为一串 char，用来保存文本信息。

使用 String 声明变量，用**双引号**包住文字：

```
String name = "Alice";
name = "Bob";
System.out.println(name); // Bob
```

String 的首字母要大写，它不属于八种基本数据类型。

**有无双引号，含义不同：**

```
int age = 18;
System.out.println(age);   // 18
System.out.println("age"); // age
```

age 读取变量的值；"age" 表示文字本身。

# 用 + 拼接字符串

+ 可以把字符串连接起来；数值参与拼接时会转换为文本。

```
String name = "Alice";
int age = 18;
String message = name + " is " + age + " years old.";
System.out.println(message); // Alice is 18 years old.
```

## 注意区分数值加法和字符串拼接：

```
System.out.println(1 + 2);           // 3
System.out.println("Total: " + 1 + 2);   // Total: 12
System.out.println("Total: " + (1 + 2)); // Total: 3
```

- 连续的 + 从左向右组合，遇到字符串就进行拼接。
- 想先算出数值结果，再拼接文字，就把计算部分放在括号中。

# 综合示例：BMI 计算需求

编写一个程序，根据身高和体重计算身体质量指数（BMI）。

- 用 `height` 保存身高，单位为米。
- 用 `weight` 保存体重，单位为千克。
- 直接修改代码中两个变量的初始值，不使用键盘输入。
- 计算 BMI，并在控制台输出带有 `BMI：` 标识的结果。

**计算公式：**体重（千克）除以身高（米）的平方。

示例数据：身高 1.75 米，体重 70 千克。

# 综合示例：BMI 计算实现

```java
public class Main {
    public static void main(String[] args) {
        double height = 1.75;
        double weight = 70.0;
        double bmi = weight / (height * height);
        System.out.println("BMI: " + bmi);
    }
}
```

- 使用 double 保存身高、体重和计算结果。
- 括号让身高先相乘，再用体重除以这个结果。
- 修改身高和体重，重新运行，就能得到新的 BMI。